

SOK-1301 - Case 1

Løsningsforslag

Derek J. Clark & Even S. Hvinden

Instruksjoner

Denne oppgaven skal løses interaktivt i RStudio ved å legge inn egen kode og kommentarer. Dette er ment som nyttig øving, og skal ikke leveres inn.

Bakgrunn

Vi skal analysere utviklingen i bruttonasjonalprodukt (BNP) i Norge. Vi bruker data Statistisk Sentralbyrå (SSB), tabell “[09842: BNP og andre hovedstørrelser \(kr per innbygger\), etter år](#)”. Tabellen inneholder årlige data på BNP per innbygger, fra 1970 til 2024. Dette notatet ble oppdatert 18. juni 2026, og antall observasjoner osv er tatt fra tabellen denne datoen.

I. API, visualisering

SSB gir oss tilgang til sine data via en [API](#) (*Application Programming Interface*), programvare som lar to applikasjoner kommunisere med hverandre. Man kan benytte R-pakken [PxWebApi-Data](#), som SSB har laget. Vi skal heller bruke en spørring som lagrer vår data som en tibble, klar til å analyseres i tidyverse.

Vi begynner med å rydde og laste inn nødvendige pakker:

```
rm(list = ls())
library(rjstat)
library(httr)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
```

```

v ggplot2 4.0.3      v tibble 3.3.1
v lubridate 1.9.5    v tidyr 1.3.2
v purrr 1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

```

NB! Du må installere `rjstat` og `httr` først. Kjør kommandoen `install.packages(c("rjstat", "httr"))` i konsollen. Det må kun gjøres én gang.

Gå til tabell “[09842: BNP og andre hovedstørrelser \(kr per innbygger\), etter år](#)”. I venstre meny velg “**Filtrer**” slik at du får opp tilgjengelige valg for denne tabellen, i dette tilfellet “Statistikkvariabel” og “År”. Jeg velger å ta “Bruttonasjonalprodukt” og “MEMO: Bruttonasjonalprodukt. Faste 2015-priser”; da har vi BNP med både løpende og faste priser. MEMO angir en variabel som er en tilleggsopplysning, ikke en del av selve regnskapet, men noe som SSB har beregnet.

The screenshot shows a web interface titled "Filtrer" (Filter) with a back arrow and "Skjul" (Hide) button. Below the title is a section "Statistikkvariabel" with an upward arrow. It indicates "2 av 6 valgt" (2 of 6 selected) and a "Må velges" (Must be selected) button. A list of variables is shown with checkboxes:

- ☐ Velg alle
- ☒ Bruttonasjonalprodukt
- ☐ Bruttonasjonalinntekt
- ☐ Nasjonalinntekt
- ☐ Disponibel inntekt for Norge
- ☐ Konsum i husholdninger og ideelle organisasjoner
- ☒ MEMO: Bruttonasjonalprodukt. Faste 2015-priser

Så haker jeg av “Velg alle” i boksen merket “År” for å få med data fra alle tilgjengelige år.

I venstremarg velger vi nå “**Lagre**”. Skroll ned til “API-spørring”, velg format som angitt nedenfor, trykk på GET, og kopier URL ved å trukke på ikonet i rødt i bildet:

API-spørring

Her finner du API-spørring for tabellen.
Bruk GET om spørringen er kort (under ca.
2 100 tegn), og POST om spørringen er
lang og mer detaljert.

[Les mer om bruk av API-et](#)

Velg format

JSON-stat2 (.json) ▼

GET

POST

GET URL



https://data.ssb.no/api/pxwebap

Våre valg av data er i denne lenken, som vi nå lagrer:

```
url <- "https://data.ssb.no/api/pxwebapi/v2/tables/09842/data?lang=no&outputFormat=json-stat2"
```

Disse url-adressene er ofte lange og du kan dele opp ved hjelp av `paste0()` på følgende vis:

```
ssb_url <- paste0(  
  "https://data.ssb.no/api/pxwebapi/v2/tables/09842/data",  
  "?lang=no",  
  "&outputFormat=json-stat2",  
  "&valuecodes[ContentsCode]=BNP,MEMOBNP",  
  "&valuecodes[Tid]=*",  
  "&heading=Tid",  
  "&stub=ContentsCode"  
)
```

I ditt Environment-vindu i RStudio bør du nå se to likens url-adresser. Vi trenger kun én, og skal bruke den som heter `url`.

Følgende kode tar dataene som vi vil hente ut, og gjør de om til en tibble som heter `df`.

```
df <- GET(url) %>%
  content(as = "text", encoding = "UTF-8") %>%
  fromJSONstat() %>%
  as_tibble()
```

La oss se på vårt datasett:

```
df
```

```
# A tibble: 110 x 3
  statistikkvariabel   år   value
  <chr>               <chr> <int>
1 Bruttonasjonalprodukt 1970  23616
2 Bruttonasjonalprodukt 1971  26363
3 Bruttonasjonalprodukt 1972  29078
4 Bruttonasjonalprodukt 1973  32805
5 Bruttonasjonalprodukt 1974  37734
6 Bruttonasjonalprodukt 1975  42884
7 Bruttonasjonalprodukt 1976  48711
8 Bruttonasjonalprodukt 1977  54652
9 Bruttonasjonalprodukt 1978  60090
10 Bruttonasjonalprodukt 1979  66069
# i 100 more rows
```

Vi ser at vi har tre kolonner med benevnelse og data type. La oss rydde i data. Det er spesielt tre ting å ta tak i:

1. Årstallene er lagret som tegn, `<chr>`. Disse skulle heller være heltall, `<int>`.
2. Kolonnenavn. “statistikkvariabel” er en lang benevnelse som vi vil erstatte, og “value” er unødvendig å skrive på engelsk. Se [3.3.3](#) `rename()` i pensumet.
3. Variabelnavn. Disse er også lang.

Oppgave 1a: Rydd i data

Skriv kode som lagrer dataene med anstendige variabelnavn og årstall som heltall. Fremover bruker jeg “var”, og “verdi” for “statistikkvariabel”, og “value”.

```
# Oppgave Ia løses her #

df <- df %>%

  rename(var = statistikkvariabel, verdi = value) %>%

  mutate(år = as.integer(år))
```

Videre erstatter vi verdiene i kolonne “var”. For å gjøre dette må vi vite nøyakig hva disse heter, for eksempel ved å kjøre:

```
df %>%
  distinct(var)
```

```
# A tibble: 2 x 1
  var
  <chr>
1 Bruttonasjonalprodukt
2 MEMO: Bruttonasjonalprodukt. Faste 2015-priser
```

Da skal vi bytte “Bruttonasjonalprodukt” med “BNP” og “MEMO: Bruttonasjonalprodukt. Faste 2015-priser” med “MEMOBNP”. Her kan vi bruke `mutate()` kombinert med `recode()` innenfor `tidyverse`.

```
df <- df %>%

  mutate(
    var = recode(
      var,
      "Bruttonasjonalprodukt" = "BNP",
      "MEMO: Bruttonasjonalprodukt. Faste 2015-priser" =
        "MEMOBNP"
    )
  )

df
```

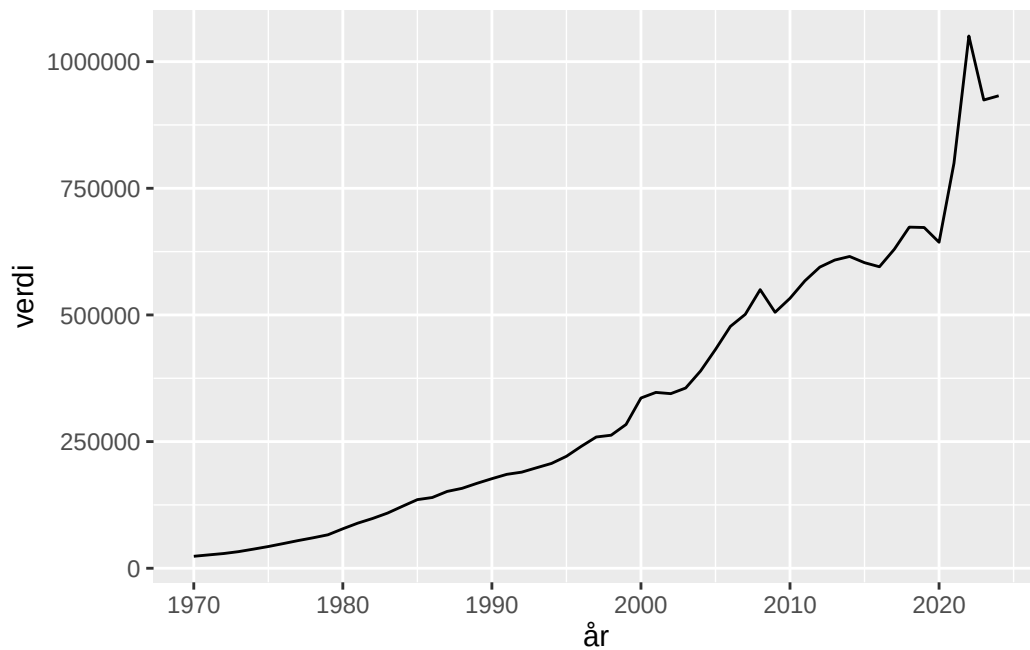
```
# A tibble: 110 x 3
  var      år verdi
  <chr> <int> <int>
```

```
1 BNP 1970 23616
2 BNP 1971 26363
3 BNP 1972 29078
4 BNP 1973 32805
5 BNP 1974 37734
6 BNP 1975 42884
7 BNP 1976 48711
8 BNP 1977 54652
9 BNP 1978 60090
10 BNP 1979 66069
# i 100 more rows
```

Oppgave lb: Lag en figur

Følgende kode skaper en enkel figur.

```
df %>%
  filter(var == "BNP") %>%
  ggplot(aes(x=år,y=verdi)) +
  geom_line()
```



Lag en pen figur som viser BNP i tusener av kroner per person, i både løpende og faste priser, mellom 2000 og 2024. Skriv en tydelig forklaring og tolkning av figuren. Hvordan har seriene

utviklet seg? Forklar forskjellen mellom BNP i løpende og faste priser. Til hvilke formål er BNP målt i faste priser relevant, og når er det riktig å se BNP i løpende priser?

```
df %>%
  mutate(verdi = verdi/1000) %>%
  filter(år >= 2000) %>%

  ggplot(aes(x=år,y=verdi, color = var)) +

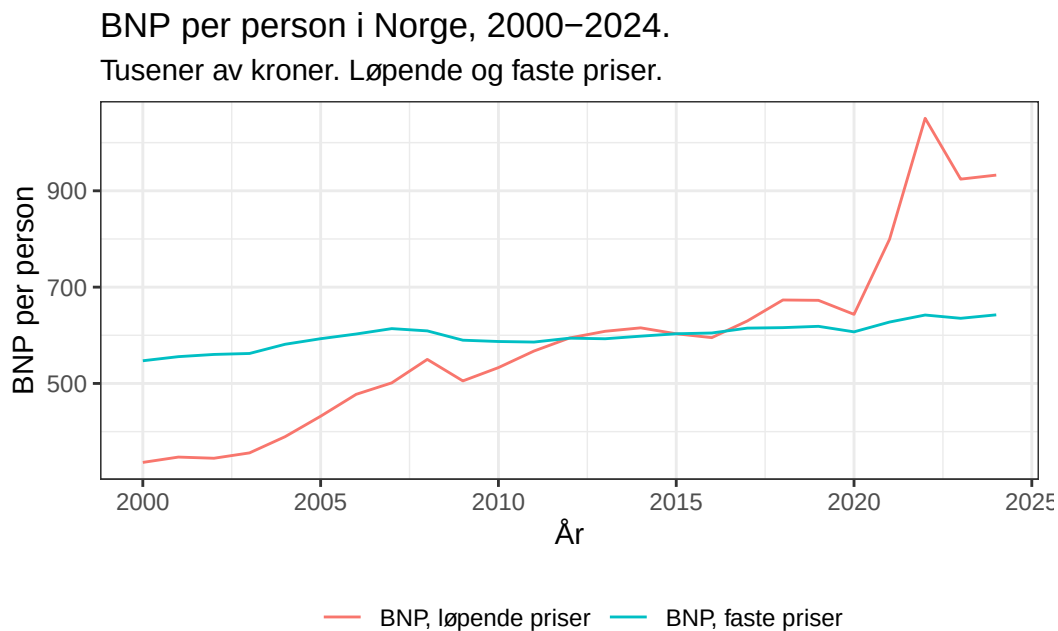
  geom_line() +

  labs(x = "År",
       y = "BNP per person",
       title = "BNP per person i Norge, 2000-2024.",
       subtitle = "Tusener av kroner. Løpende og faste priser.") +

  scale_color_discrete(name = " ",
                       labels = c("BNP, løpende priser", "BNP, faste priser")) +

  theme_bw() +

  theme(legend.position = "bottom")
```



Her har jeg endret tegnforklaringen ved å bruke `scale_color_discrete()` . `name = " utelater variabelnavn ("var"), og labels = c("BNP, løpende priser", "BNP, faste priser") endrer teksten fra "BNP" og "MEMOBNP". Rekkefølgen er viktig her - "var" er en vanlig tekststreng her, og da arrangerer R disse vanligvis alfabetisk, dvs BNP før MEMOBNP. For å sjekke kan man skrive:`

```
sort(unique(df$var))
```

```
[1] "BNP"      "MEMOBNP"
```

som gir rekkefølgen som brukes i `labels()` .

Figuren viser nivået på BNP per person, i tusener av kroner, fra 2000 til 2024. Den røde linjen viser BNP målt med løpende priser, og kalles også nominell BNP. Endringen i nominell BNP er gitt ved både verdiskapning og prisene. Den blå linjen viser BNP med faste 2015-priser, og kalles reell BNP. Endringen i reell BNP måler kun endringen i verdiskapning. Per definisjon er nominelt og reelt BNP like i 2015. Vi ser at reelt BNP har holdt seg stabil rundt 600 000 kroner per person siden 2005, mens nominelt BNP har vokst kraftig.

Nominell BNP er relevant i sammenligning med nominelle størrelser, for eksempel skatteinntekter, offentlige utgifter eller gjeld som andel av BNP. Reell BNP viser den faktiske utviklingen i kjøpekraft og er den relevante størrelsen for diskusjoner om velferd.

Faste priser svarer best på spørsmålet «har vi produsert mer?», mens løpende priser svarer på «hva er produksjonen verdt i dagens priser?»

II. Transformasjon, visualisering

Våre data er en tidsserie, hvilket betyr at rekkefølgen i observasjonene er ordnet etter tid. Vi skal nå regne prosentvis, årlig endring. La x_t være BNP i år t . For eksempel vil x_{1970} være 23616.

Den årlige endringen i BNP fra år $t - 1$ til t er gitt ved $x_t - x_{t-1}$. I samfunnsøkonomi er det vanlig å betegne dette som $\Delta x_t := x_t - x_{t-1}$. Tegnet Δ er den greske bokstaven delta og betegner differanse. For eksempel vil $\Delta x_{1971} = 26363 - 23616 = 2747$ kroner.

I mange tilfeller er vi interesserte i relativ vekst: Hvor mye økte BNP, relativt til hva den var i utgangspunkt? Den mest brukte enheten er hundredeler eller prosentvis endring, gitt ved $100 \times \Delta x_t / x_{t-1}$. For eksempel var den prosentvise endringen i BNP i 1971 $100 \times \Delta x_{1971} / x_{1970} = 100 \times (2747 / 23616) \approx 11.6$, hvor $\approx$ betegner "omtrent lik" da jeg viser svaret med kun én desimal. Tilsvarende kan man skrive at $\Delta x_{1971} / x_{1970} = 2747 / 23616 \approx 0.116 = 11.6\%$, hvor tegnet % betegner at beløpet oppgis i hundredeler eller prosent.

Oppgave IIa: Omorganisere datasett med `pivot_wider()`

Vi skal lage to variable `DBNP` og `DMEMOBNP` som viser relativ endring i `BNP` og `MEMOBNP`. Til dette formålet skal vi bruke kommandoene `pivot_wide()` og `pivot_long()` til å omorganisere dataene. Jeg anbefaler dere først å lese [kapittel 5.3/5.4](#) i pensum. Betrakt følgende kode.

```
df_wide <- df %>%  
  pivot_wider(names_from = var, values_from = verdi)  
  
df_wide
```

```
# A tibble: 55 x 3  
   år    BNP MEMOBNP  
  <int> <int>   <int>  
1  1970 23616  215602  
2  1971 26363  226242  
3  1972 29078  236487  
4  1973 32805  245483  
5  1974 37734  253534  
6  1975 42884  264625  
7  1976 48711  278730  
8  1977 54652  289103  
9  1978 60090  299145  
10 1979 66069  311164  
# i 45 more rows
```

Beskriv konkret hva koden gjorde. Sammenlign `df` og `df_wide`.

Koden omorganiserte datasettet. I utgangspunktet er `BNP` og `MEMOBNP` verdier av variabelen `var`. Verdien av variablene var lagret i variabelen `verdi`. Kommandoen `pivot_wider` lager to nye variable med variabelnavn fra `var` og verdier fra `verdi`. Vi ser derfor at `df_wide` har 55 observasjoner av tre variable (`år`, `BNP`, og `MEMOBNP`) mens `df` har 110 observasjoner av tre variable (`år`, `var`, og `verdi`).

Oppgave IIb: Beregn vekst

Til å beregne endring er funksjonen `lag()` meget nyttig. I denne konteksten er begrepet *lag* et engelsk verb som beskriver foregående observasjon. Bruker vi funksjonen `lag()` på en variabel (kolonne) så returnerer den en ny kolonne hvor verdien er lik foregående observasjon. Se [pensum 13.5.2](#). Betrakt følgende kode:

```
df_wide <- df_wide %>%
  mutate(LBNP = lag(BNP)) %>%
  mutate(LMEMOBNP = lag(MEMOBNP))

# legger variablene i rekkefølge

df_wide <- df_wide %>%
  relocate(LBNP, .before = MEMOBNP)

df_wide
```

```
# A tibble: 55 x 5
   år   BNP  LBNP MEMOBNP LMEMOBNP
<int> <int> <int>   <int>   <int>
1  1970 23616    NA  215602     NA
2  1971 26363 23616  226242  215602
3  1972 29078 26363  236487  226242
4  1973 32805 29078  245483  236487
5  1974 37734 32805  253534  245483
6  1975 42884 37734  264625  253534
7  1976 48711 42884  278730  264625
8  1977 54652 48711  289103  278730
9  1978 60090 54652  299145  289103
10 1979 66069 60090  311164  299145
# i 45 more rows
```

Hvis vi bruker den matematiske notasjonen diskutert tidligere så har nå kolonner med x_t (BNP, MEMOBNP) og x_{t-1} (LBNP, LMEMOBNP). Bruk funksjonen `mutate()` til å lage en ny variabel med relativ endring i BNP og MEMOBNP i `df_wide`. Kall disse DBNP og DMEMOBNP.

```
df_wide <- df_wide %>%
  mutate(DBNP = (BNP-LBNP)/LBNP) %>%
  mutate(DMEMOBNP = (MEMOBNP-LMEMOBNP)/LMEMOBNP)

df_wide
```

```
# A tibble: 55 x 7
   år   BNP  LBNP MEMOBNP LMEMOBNP   DBNP DMEMOBNP
<int> <int> <int>   <int>   <int>   <dbl>   <dbl>
1  1970 23616    NA  215602     NA    NA      NA
2  1971 26363 23616  226242  215602  0.116  0.0494
3  1972 29078 26363  236487  226242  0.103  0.0453
```

```

4  1973 32805 29078 245483 236487 0.128 0.0380
5  1974 37734 32805 253534 245483 0.150 0.0328
6  1975 42884 37734 264625 253534 0.136 0.0437
7  1976 48711 42884 278730 264625 0.136 0.0533
8  1977 54652 48711 289103 278730 0.122 0.0372
9  1978 60090 54652 299145 289103 0.0995 0.0347
10 1979 66069 60090 311164 299145 0.0995 0.0402
# i 45 more rows

```

Oppgave IIc: Omorganisere datasett med pivot_longer()

Bruk nå funksjonen `pivot_longer()` til å transformere `df_wide` til det opprinnelige formatet, altså med variablene `var` og `verdi`. Kall den transformerte tabellen for `df_long`.

NB! I `names_to` og `values_to` skriver vi navnene på de nye variablene med anførselstegn, for eksempel `names_to = "var"` og `values_to = "verdi"`.

```

df_long <- df_wide %>%
  pivot_longer(
    cols = !år,
    names_to = "var",
    values_to = "verdi",
    values_drop_na = TRUE
  )

```

```
df_long
```

```

# A tibble: 326 x 3
   år var      verdi
  <int> <chr>    <dbl>
1  1970 BNP      23616
2  1970 MEMOBNP 215602
3  1971 BNP      26363
4  1971 LBNP     23616
5  1971 MEMOBNP 226242
6  1971 LMEMOBNP 215602
7  1971 DBNP      0.116
8  1971 DMEMOBNP 0.0494
9  1972 BNP      29078
10 1972 LBNP     26363

```

```
# i 316 more rows
```

```
# alternativt

df_long <- df_wide %>%
  pivot_longer(
    cols = 2:7,
    names_to = "var",
    values_to = "verdi",
    values_drop_na = TRUE
  )

# eller

df_long <- df_wide %>%
  pivot_longer(
    cols = BNP:DMEBOBNP,
    names_to = "var",
    values_to = "verdi",
    values_drop_na = TRUE
  )
```

Oppgave IId: Figur med vekst

Lag en pen figur med prosentvis vekst i nominelt og reelt BNP per person fra 1970 til 2024. Finnes det observasjoner med negativ vekst i reell BNP? Hva skyldes dette?

Merknad: Det er en del støy i data. Prøv å kombinere `geom_point()` og `geom_smooth()` for å få et bedre inntrykk av den langsiktige utviklingen.

```
df_long %>%
  mutate(verdi = verdi*100) %>%

  filter(var == c("DBNP","DMEBOBNP")) %>%

  ggplot(aes(x=år, y=verdi, color = var)) +

  geom_point() +

  geom_smooth(method = "loess", se = FALSE) +

  labs(x = "År",
```

```

y = "BNP per person",
title = "BNP per person i Norge, 1970-2024.",
subtitle = "Årsvekst, prosent. Løpende og faste priser.") +

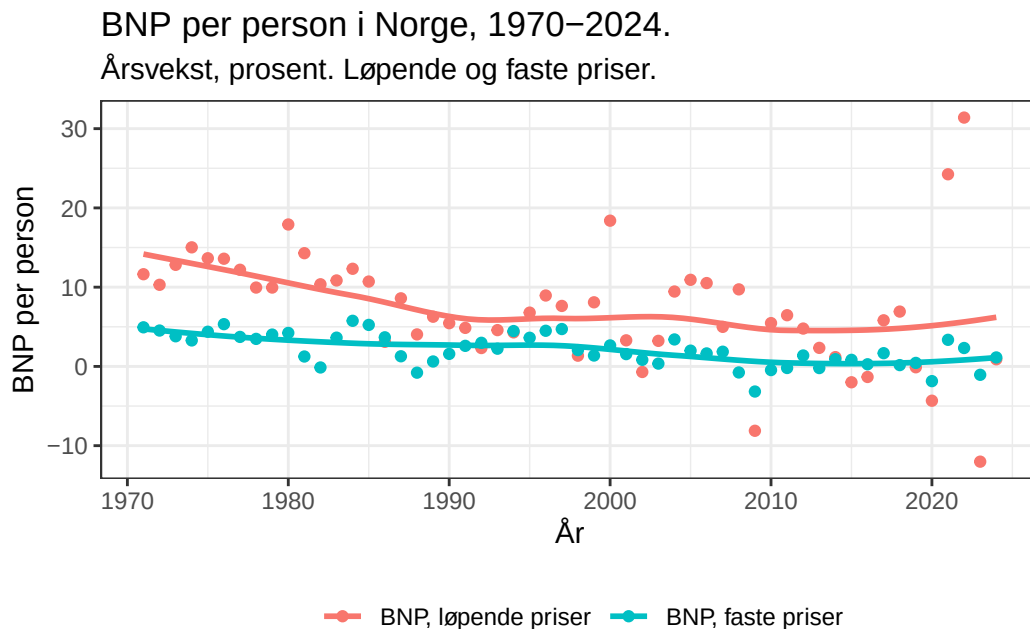
scale_color_discrete(name = " ",
                      labels = c("BNP, løpende priser", "BNP, faste priser")) +

theme_bw() +

theme(legend.position = "bottom")

```

`geom\_smooth()` using formula = 'y ~ x'



`method = "lm"` gir én rett linje som oppsummerer det gjennomsnittlige forholdet over hele perioden 1970-2024. Dersom sammenhengen endrer seg over tid, kan én rett linje skjule viktige variasjoner. `method = "loess"` tilpasser i stedet en glatt kurve basert på lokale deler av datasettet, og kan derfor gi et bedre bilde av hvordan utviklingen har endret seg over tid.